In [4]:
```python
import math
import random
import re
import os
from collections import defaultdict, Counter



def load_corpus(filepath: str) -> list:
    """
    Load and tokenise The Office corpus from a plain text file.
    Returns a list of lowercase word tokens.
    Expects one line per utterance/sentence in the file.
    """
    tokens = []
    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip().lower()
            if not line:
                continue
            # Basic tokenisation: split on whitespace, strip punctuation
            words = re.findall(r"[a-z']+", line)
            if words:
                tokens.extend(["<START>"] + words + ["<END>"])
    return tokens


def train_test_split(tokens: list, test_ratio: float = 0.1):
    """Split token list into train and test portions."""
    split = int(len(tokens) * (1 - test_ratio))
    return tokens[:split], tokens[split:]



class BigramLM:
    """
    Bigram Language Model with Laplace (add-1) smoothing.

    The Markov assumption here: P(wi | w1...wi-1) ≈ P(wi | wi-1)
    Each word depends only on the single preceding word — a first-order
    Markov chain. This is efficient and interpretable but fundamentally
    cannot capture dependencies beyond one step.
    """

    def __init__(self):
        self.bigram_counts = defaultdict(Counter)
        self.unigram_counts = Counter()
        self.vocab = set()
        self.log_probs = defaultdict(dict)

    def train(self, tokens: list):
```

```
        """
        Estimate bigram transition probabilities P(wi | wi-1) via MLE
        with Laplace smoothing.
        """
        for i in range(len(tokens) - 1):
            w1, w2 = tokens[i], tokens[i + 1]
            self.bigram_counts[w1][w2] += 1
            self.unigram_counts[w1] += 1
            self.vocab.add(w1)
            self.vocab.add(w2)

        # P(w2 | w1) = (count(w1, w2) + 1) / (count(w1) + |V|)
        V = len(self.vocab)
        for w1 in self.bigram_counts:
            total = self.unigram_counts[w1]
            for w2 in self.vocab:
                count = self.bigram_counts[w1][w2]
                self.log_probs[w1][w2] = math.log((count + 1) / (total + V))

    def log_prob(self, w1: str, w2: str) -> float:
        """Return log P(w2 | w1) with fallback for unseen context."""
        if w1 in self.log_probs and w2 in self.log_probs[w1]:
            return self.log_probs[w1][w2]
        return math.log(1 / len(self.vocab)) if self.vocab else float('-inf')


    def generate(self, max_len: int = 20) -> str:
        """
        Generate a sentence by sampling from the learned bigram distribution.
        Starts from <START> and samples until <END> or max_len is reached.
        """
        word = "<START>"
        sentence = []
        for _ in range(max_len):
            if word not in self.log_probs:
                break
            candidates = list(self.log_probs[word].items())
            words, log_ps = zip(*candidates)
            ps = [math.exp(lp) for lp in log_ps]
            total = sum(ps)
            ps = [p / total for p in ps]
            word = random.choices(words, weights=ps, k=1)[0]
            if word == "<END>":
                break
            sentence.append(word)
        return " ".join(sentence)



    def perplexity(self, tokens: list) -> float:
        """
        Compute perplexity on a token sequence.
        PP = exp(-1/N * sum(log P(wi | wi-1)))
```

```
        Lower perplexity = better model fit.
        """
        log_prob_sum = 0.0
        N = 0
        for i in range(len(tokens) - 1):
            w1, w2 = tokens[i], tokens[i + 1]
            if w2 == "<START>":
                continue
            log_prob_sum += self.log_prob(w1, w2)
            N += 1
        return math.exp(-log_prob_sum / N) if N > 0 else float('inf')




corpus_path = None
for root, dirs, files in os.walk("/kaggle/input"):
    for f in files:
        if f.endswith(".txt"):
            corpus_path = os.path.join(root, f)
            break

if corpus_path is None:
    raise FileNotFoundError("No .txt file found under /kaggle/input. Please up

print(f"Using corpus: {corpus_path}")

tokens = load_corpus(corpus_path)
print(f"Total tokens: {len(tokens)}")

train_tokens, test_tokens = train_test_split(tokens, test_ratio=0.1)
print(f"Train tokens: {len(train_tokens)} | Test tokens: {len(test_tokens)}")

model = BigramLM()
model.train(train_tokens)
print(f"Vocabulary size: {len(model.vocab)}")

print("\n--- 5 Generated Sentences ---")
for i in range(5):
    print(f"  {i+1}: {model.generate()}")

ppl = model.perplexity(test_tokens)
print(f"\nTest Set Perplexity: {ppl:.4f}")

# Sanity check
phrase = ["<START>", "that", "is", "what", "she", "said", "<END>"]
print(f"Perplexity of 'that is what she said': {model.perplexity(phrase):.4f}"
```

```
Using corpus: /kaggle/input/datasets/shauryahandu/python-bigram-lm-py-the-office-txt/office_script_clean (1).txt
Total tokens: 738231
Train tokens: 664407 | Test tokens: 73824
Vocabulary size: 18673

--- 5 Generated Sentences ---
  1: pete hahahaha neighbor appearances mechanic many amicable moan jaws knot successfully quieter tumor eczema breastbone ryan damn workforce impressive skulls
  2: michael eczema lad inches federer serves runner's cuts yeppers congratulate fatherson preparedness politicians remove boys' holder spaghetti el affirmation freak
  3: kelly corndogs libby vibrate fires perception deb thankfully diligent swan blob successories clookies counting behaved nba's appendicitis 'impressions running circuit
  4: karen momentum hoo dynamite barfo road 'd argument plait plugs fogg's happens dorm punishment hardcore goes yehhh blindfold backs wasting
  5: michael i have sample patrollers signature coach coding applicants me now snowing squash hup brittle fruit impressed tankard mets gorilla

Test Set Perplexity: 1324.0627
Perplexity of 'that is what she said': 623.9403
```

In [ ]: